

Searching Algorithms

K. M. Safin Kamal

Lecturer

CSE, EWU

Linear Search Algorithm

A fundamental sequential searching technique in Data Structures

What is Linear Search?

The Concept

Linear search, also known as sequential search, is the simplest method for finding an element within a list or array.

It involves iterating over all elements of the array one by one and checking if the current element matches the target element (the key).

The Outcome

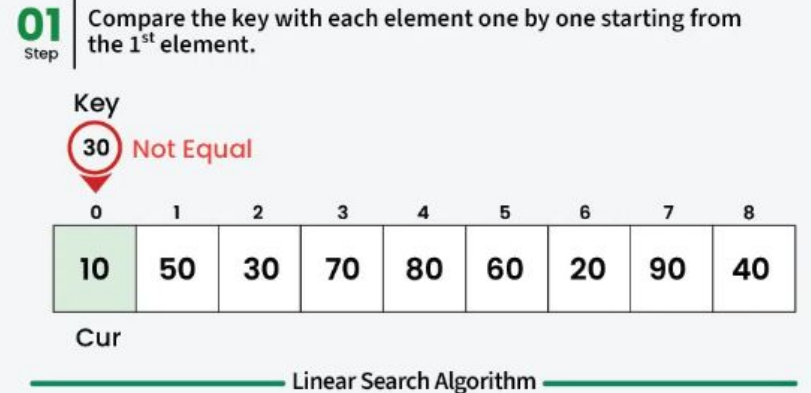
- ▶ **Match Found:** If an element is found to be equal to the target, the algorithm returns the index of that current element.
- ▶ **No Match:** If the loop finishes and no element matches the target, it returns -1, indicating the element is not present.

Visualizing the Process

Example Scenario

Consider an array `arr[]` and a target $= x$.

- ▶ Start from the leftmost element of `arr[]`.
- ▶ Compare x with each element.
- ▶ If x matches an element, return the index.
- ▶ If the end of the array is reached without a match, return -1.



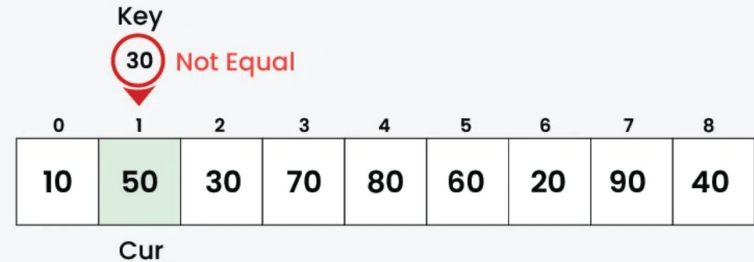
Visualizing the Process

Example Scenario

Consider an array `arr[]` and a target $= x$.

- ▶ Start from the leftmost element of `arr[]`.
- ▶ Compare x with each element.
- ▶ If x matches an element, return the index.
- ▶ If the end of the array is reached without a match, return -1.

02 Step | Compare the key with 2nd element which is not equal to the key, so move to the next element



Linear Search Algorithm

Visualizing the Process

Example Scenario

Consider an array `arr[]` and a target $= x$.

- ▶ Start from the leftmost element of `arr[]`.
- ▶ Compare x with each element.
- ▶ If x matches an element, return the index.
- ▶ If the end of the array is reached without a match, return -1.

03
Step

Compare the key with the 3rd element. Key is found so stop the search.

Key								
30 Equal								
0	1	2	3	4	5	6	7	8
10	50	30	70	80	60	20	90	40

Cur

Linear Search Algorithm

Iterative Implementation Example (C++)

```
// Function to perform linear search
int linearSearch(int arr[], int size, int target) {
    // Iterate through each element in the array
    for (int i = 0; i < size; i++) {
        // If the current element matches the target, return its index
        if (arr[i] == target) {
            return i;
        }
    }
    // If the loop finishes without finding the target, return -1
    return -1;
}
```

Recursive Implementation Example (C++)

```
// Function to perform Recursive Linear Search
int recursiveLinearSearch(int arr[], int size, int currentIndex, int target) {
    // Base Case 1: We reached the end of the array without finding the target
    if (currentIndex == size) {
        return -1;
    }

    // Base Case 2: The current element matches the target
    if (arr[currentIndex] == target) {
        return currentIndex;
    }

    // Recursive Case: Call the function again, moving 1 index to the right
    return recursiveLinearSearch(arr, size, currentIndex + 1, target);
}

// Example usage:
// int result = recursiveLinearSearch(myArray, size, 0, target);
```

Complexity Analysis

Time Complexity

Best Case:

$O(1)$

The key is present at the first index.

Worst Case:

$O(N)$

The key is at the last index or not present at all. (N is the size of the list).

Average Case:

$O(N)$

Space Complexity (Iterative)

Auxiliary Space:

$O(1)$

Linear search is highly memory efficient. Except for the iterator variable, no additional space is required regardless of the input size.

Space Complexity (Recursive)

Auxiliary Space:

$O(N)$

Applications & Use Cases



Unsorted Data

When dealing with an unsorted array or list, linear search is the standard, most commonly used method to find an element.



Small Datasets

For very small datasets, Linear Search is often preferred over Binary Search because the overhead of sorting or complex logic isn't justified.



Linked Lists

In linked list implementations, elements cannot be accessed randomly. Linear search sequentially checks each node until the element is found.

Binary Search Algorithm

An efficient divide-and-conquer search technique

Core Principles

Prerequisites

Unlike Linear Search, Binary Search has strict requirements to function correctly:

- ▶ **Sorted Data:** The data structure (like an array) *must* be sorted beforehand (either ascending or descending).
- ▶ **Constant Access:** It requires random access to elements in constant time (e.g., arrays, where you can access any index directly).

The Mechanism

It operates by repeatedly dividing the search space in half:

- ▶ Find the middle index mid.
- ▶ Compare the middle element with the target key.
- ▶ If a match is found, terminate.
- ▶ If the key is smaller, discard the right half.
- ▶ If the key is larger, discard the left half.

Visualizing Divide & Conquer

Finding the Middle

To avoid integer overflow issues in large arrays, the middle index is typically calculated safely as:

$$\text{mid} = \text{low} + \frac{(\text{high} - \text{low})}{2}$$

With each comparison, the algorithm eliminates half of the remaining possibilities, making it exceptionally fast for large datasets.

Initially

Find Key = 23 using Binary Search

	0	1	2	3	4	5	6	7	8	9
arr[] =	2	5	8	12	16	23	38	56	72	91

Visualizing Divide & Conquer

Finding the Middle

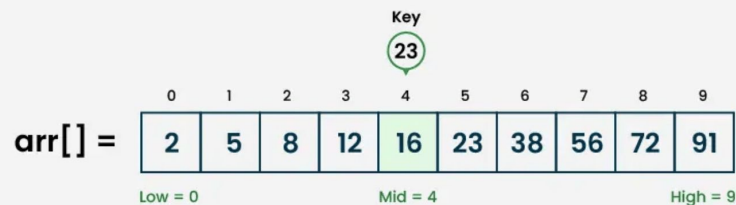
To avoid integer overflow issues in large arrays, the middle index is typically calculated safely as:

$$\text{mid} = \text{low} + \frac{(\text{high} - \text{low})}{2}$$

With each comparison, the algorithm eliminates half of the remaining possibilities, making it exceptionally fast for large datasets.

Step 1

Key (i.e., 23) is greater than current mid element (i.e., 16). The search space moves to the right.



Visualizing Divide & Conquer

Finding the Middle

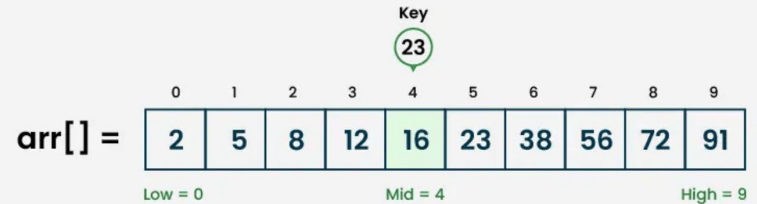
To avoid integer overflow issues in large arrays, the middle index is typically calculated safely as:

$$\text{mid} = \text{low} + \frac{(\text{high} - \text{low})}{2}$$

With each comparison, the algorithm eliminates half of the remaining possibilities, making it exceptionally fast for large datasets.

Step 1

Key (i.e., 23) is greater than current mid element (i.e., 16). The search space moves to the right.



Visualizing Divide & Conquer

Finding the Middle

To avoid integer overflow issues in large arrays, the middle index is typically calculated safely as:

$$\text{mid} = \text{low} + \frac{(\text{high} - \text{low})}{2}$$

With each comparison, the algorithm eliminates half of the remaining possibilities, making it exceptionally fast for large datasets.

Step 3

If the key matches the value of the mid element, the element is found and stop search.



Iterative Implementation (C++)

```
int binarySearch(int arr[], int size, int target) {
    int low = 0;
    int high = size - 1;

    // Loop until the search space is exhausted
    while (low <= high) {
        // Safely calculate the middle index to prevent overflow
        int mid = low + (high - low) / 2;

        // Check if the target is present at the middle
        if (arr[mid] == target) {
            return mid;
        }

        // If the target is greater than the middle element,
        // ignore the left half
        if (arr[mid] < target) {
            low = mid + 1;
        }

        // If the target is smaller than the middle element,
        // ignore the right half
        else {
            high = mid - 1;
        }
    }

    return -1;
}
```

Recursive Implementation (C++)

```
// Function to perform Recursive Binary Search
int recursiveBinarySearch(int arr[], int low, int high, int target) {
    // Base Case 1: The search space is exhausted (target not found)
    if (low > high) {
        return -1;
    }

    // Safely calculate the middle index to prevent overflow
    int mid = low + (high - low) / 2;

    // Base Case 2: The target is found at the middle
    if (arr[mid] == target) {
        return mid;
    }

    // Recursive Case 1: Target is smaller than mid, so search the left half
    if (arr[mid] > target) {
        return recursiveBinarySearch(arr, low, mid - 1, target);
    }

    // Recursive Case 2: Target is greater than mid, so search the right half
    return recursiveBinarySearch(arr, mid + 1, high, target);
}

// Example usage:
// int result = recursiveBinarySearch(myArray, 0, size - 1, target);
```

Complexity Analysis

Time Complexity

Best Case:

$O(1)$

The key is exactly at the middle index on the first check.

Worst/Avg Case:

$O(\log N)$

The search space is halved repeatedly until the element is found or the space is exhausted.

Space Complexity (Iterative)

Auxiliary Space:

$O(1)$

The iterative method only uses a few pointers (low, high, mid), making it highly memory efficient.

Space Complexity (Recursive)

Auxiliary Space:

$O(\log N)$

The recursive method requires space on the call stack proportional to the maximum depth of recursion.

When to Use Binary Search?



Sorted Datasets

It is the de-facto standard for finding elements in sorted arrays, lists, or database indexes due to its logarithmic speed.



Large Data Volumes

While Linear Search takes 1 million steps for 1 million items, Binary Search takes at most 20 steps for the same dataset.



Optimization Problems

Often used in algorithmic problem-solving as "Binary Search on Answer" to find optimal values in a monotonic search space.

Complexity Mathematics

A rigorous mathematical proof of Time and Space complexities for Linear vs.
Binary Search algorithms.

Math of Linear Search

Worst-Case Scenario

In Big-O notation, we evaluate the worst-case scenario. For a linear search, this occurs when the target is at the very end of the array, or missing entirely.

If we have an array of size n , the algorithm must check every single element exactly once.

Step 1: 1 comparison

Step 2: 1 comparison

...

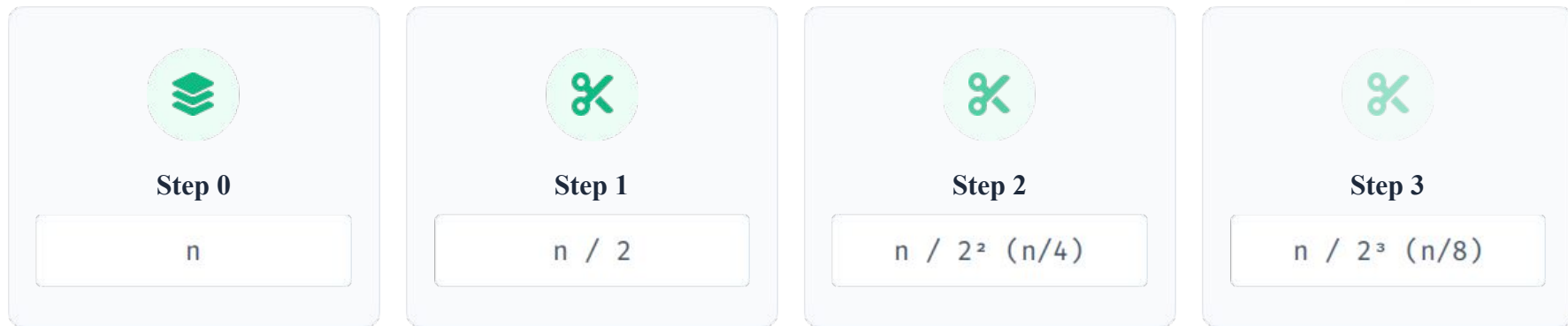
Step n : 1 comparison

Mathematically, operations $T(n)$ are directly proportional to size n :

$$T(n) = n$$

Dividing the Search Space

Binary search mathematically divides the search space in half. Let k be the total number of steps required to reach the worst-case scenario (1 element remaining).



By observing the geometric progression, after k steps, the remaining search space is: $n / 2^k$

Deriving $O(\log n)$

Solving for k

In the worst case, we keep dividing until the search space is exactly 1 element. We set up an algebraic equation to solve for k .

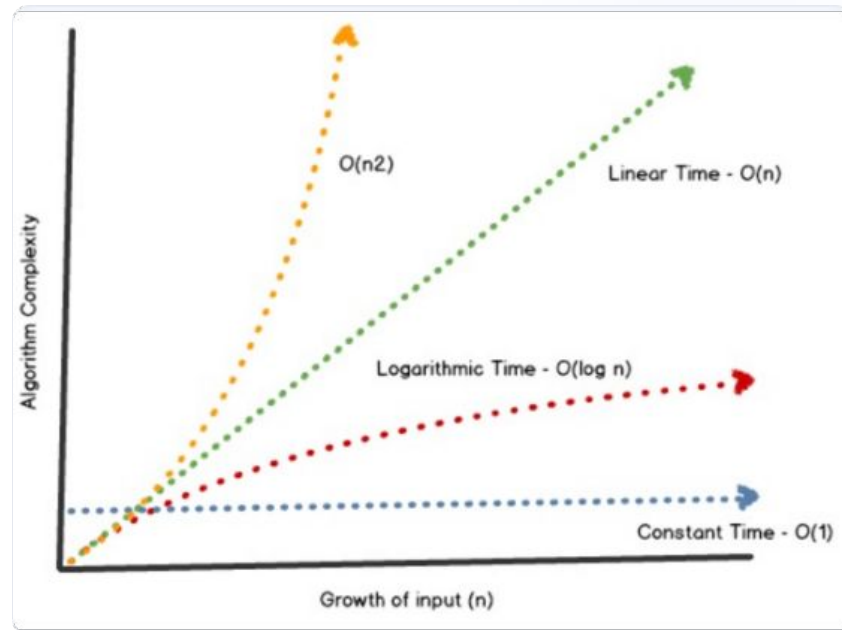
- ▶ **Isolate n :** Multiply both sides by 2^k .
- ▶ **Extract Exponent:** Take the base-2 logarithm of both sides.
- ▶ **Simplify:** Applying log rules, the exponent becomes our time complexity.

$$\begin{aligned}\frac{n}{2^k} &= 1 \\ n &= 2^k \\ \log_2(n) &= \log_2(2^k) \\ k &= \log_2(n)\end{aligned}$$

Visualizing the Difference

Exponential Flattening

- ▶ **Linear Growth:** Climbs steadily. An array of 100 elements takes 100 operations.
- ▶ **Logarithmic Growth:** Flattens out drastically. Because it cuts the workload in half every step, 100 elements take only ≈ 6.64 operations.
- ▶ **At Scale:** If expanded to 1,000,000 elements, Linear hits 1,000,000 ops, while Binary takes merely 20 operations!



Space Complexity Analysis

While Time Complexity remains identical between iterative and recursive methods, Space Complexity (Auxiliary Memory) changes dramatically due to the **call stack**.

Algorithm	Implementation	Time Complexity	Space Complexity
Linear Search	Iterative (Loop)	$O(n)$	$O(1)$
Linear Search	Recursive	$O(n)$	$O(n)$
Binary Search	Iterative (Loop)	$O(\log n)$	$O(1)$
Binary Search	Recursive	$O(\log n)$	$O(\log n)$